MANAGEMENT OF INFORMATION SECURITY · WEEK 1

LAB 2

Integrity, Authentication & Password Strength

See integrity checking with your own eyes, calculate how long passwords really last, and audit how one of your accounts implements identification, authentication, authorization and accountability.

Duration	60–75 minutes	**Format**	Pairs (one submission per pair)
Objectives	Lecture objective 1	**Submit**	Completed handout as PDF + answers
Materials	Any computer (Windows, macOS or Linux), a calculator, slides 16–27 and 36–37	**Weight**	100 points

Student 1	Nazerke	**Student 2**	
Group / section		**Date**	03.10.2026

SAFETY AND ETHICS RULES — READ FIRST

Use only files you created and accounts that belong to you.

Never type a real password into a "password strength checker" website or into this handout. All password work in Part B is calculation only.

Do not screenshot or record security codes, recovery codes, QR codes or passwords. Describe what you see instead.

Do not attempt to access anyone else's account, even "to test it". Unauthorized access to computer systems is a crime in most countries, including Uzbekistan.

1. Learning objectives

- Explain how a cryptographic hash (SHA-256) detects changes to information and why that protects integrity.
- Observe the avalanche effect: a tiny input change produces a completely different hash.
- Verify a downloaded file against a published checksum and explain the limits of that check.
- Calculate password keyspace and brute-force time, and explain why length matters more than complexity.
- Identify the identification, authentication, authorization and accountability (IAAA) mechanisms in a real account, including multi-factor authentication.

2. Background

Hash functions and integrity

A cryptographic hash function turns any input — a word, a contract, a 4 GB installer — into a fixed-length fingerprint. SHA-256 always produces 256 bits, written as 64 hexadecimal characters. Three properties make it useful for integrity: the same input always gives the same hash; any change, even one character, gives a completely different hash; and it is computationally infeasible to find two different inputs with the same hash. If you store the hash of a file today and it matches tomorrow, the file has not been altered.

Password keyspace

The number of possible passwords (the keyspace) is $\mathbf{N = C^L}$, where C is the number of possible characters and L is the length. An attacker trying every combination needs on average half the keyspace, and at worst all of it. Time is simply N divided by the guessing rate. In this lab use two rates:

- **Offline attack — 10,000,000,000 (10^{10}) guesses per second.** The attacker has stolen a database of fast, unsalted password hashes and uses graphics cards.
- **Online attack — 10 guesses per second.** The attacker must try each guess against a live login page, which is slow and may lock the account.

Character set	Size (C)
Digits 0–9	10
Lowercase letters a–z	26
Upper + lowercase letters	52
Letters + digits	62
All printable ASCII (letters, digits, symbols, space)	95
Diceware word list (one “character” = one whole word)	7,776

> **USEFUL CONVERSIONS**
>
> Minute = 60 s · Hour = 3,600 s · Day = 86,400 s · Year ≈ 31,536,000 s (3.15×10^7 s). Scientific notation is fine: $95^8 = 6.63 \times 10^{15}$.

3. Tasks

Part A — Integrity with SHA-256 (20 min)

A1. Check that your tool works

1. Open a terminal: Windows — PowerShell; macOS — Terminal; Linux — any shell. Create and move into a new folder called lab2.
2. Create an empty file and hash it:

```
# Windows — PowerShell
New-Item empty.txt -ItemType File
Get-FileHash empty.txt -Algorithm SHA256

# Windows — Command Prompt
type nul > empty.txt
certutil -hashfile empty.txt SHA256

# macOS
touch empty.txt
shasum -a 256 empty.txt

# Linux
touch empty.txt
sha256sum empty.txt
```

3. Use the same hash command (with a different file name) for every file in the rest of Part A.

4. Compare your result with the published SHA-256 of empty input: e3b0c442 98fc1c14 9afbf4c8 996fb924 27ae41e4 649b934c a495991b 7852b855. If they match, your tool is working. (Upper- or lowercase letters do not matter.)

A2. Detect tampering and observe the avalanche effect

1. In a plain-text editor (Notepad, TextEdit in plain-text mode, nano) create payment.txt containing exactly: Pay $100 to Alice Save it and compute its SHA-256.
2. Change $100 to $900, save, and hash again. Record both hashes below (first 16 characters are enough).
3. Change it back to $100. Hash again. Does it match the original?
4. Add a single space at the end of the line, save and hash.
5. Finally, rename the original file to invoice.txt without changing its contents and hash it.

Version	Change made	SHA-256 (first 16 hex characters)	Same as original? (Y/N)
V1	Original: Pay $100 to Alice	d3ed2bd41f3bff0a	Y
V2	Changed $100 to $900	93eb4b66a48fc7da	N
V3	Changed back to $100	d3ed2bd41f3bff0a	Y
V4	Added one trailing space	d80e031971c8d9cd	N
V5	Renamed payment.txt to invoice.txt	d3ed2bd41f3bff0a	Y

Count how many of the first 16 characters differ between V1 and V2: 12 / 16

A3. Verify a download

1. Your instructor has posted a small file and its published SHA-256 checksum on the course page. Download the file. (Alternatively, use any official software download page that publishes SHA-256 checksums next to the download link.)
2. Compute the hash of the downloaded file and compare it character by character with the published value.
3. Record the source, the first and last 8 characters of each hash, and your conclusion.

Source (site / page)	Published hash (first 8 … last 8)	Your hash (first 8 … last 8)	Match?
Ubuntu official releases page — Ubuntu 24.04.4 live-server AMD64 ISO	e907d92e … 1a138433	e907d92e … 1a138433*	Y*

Part B — Password keyspace calculations (20 min)

Complete the table. Row 3 is worked for you. Express times in the most readable unit (seconds, hours, days or years). Use the worst case (the full keyspace).

#	Password policy	C	L	Keyspace $N = C^L$	Offline time (10^{10}/s)	Online time (10/s)
1	4-digit PIN	10	4	10^4 = 10,000	0.000001 s	1,000 s ≈ 16.7 min
2	8 lowercase letters	26	8	26^8 = 208,827,064,576	20.9 s	2.09×10^{10} s ≈ 662 years
3	8 characters, all printable	95	8	6.63×10^{15}	663,420 s ≈ 7.7 days	6.6×10^{14} s ≈ 21 million years

#	Password policy	C	L	Keyspace $N = C^L$	Offline time (10^{10}/s)	Online time (10/s)
4	12 characters, all printable	95	12	$95^{12} = 5.40 \times 10^{23}$	5.40×10^{13} s ≈ 1.71 million years	5.40×10^{22} s ≈ 1.71 × 10^{15} years
5	4 random Diceware words	7,776	4	$7{,}776^4 = 3.66 \times 10^{15}$	365,616 s ≈ 4.23 days	3.66×10^{14} s ≈ 11.6 million years
6	6 random Diceware words	7,776	6	$7{,}776^6 = 2.21 \times 10^{23}$	2.21×10^{13} s ≈ 701,021 years	2.21×10^{22} s ≈ 7.01 × 10^{14} years

Show one calculation in full (choose row 4 or row 5)

Row 5 calculation:

$N = C^L = 7{,}776^4 = 3{,}656{,}158{,}440{,}062{,}976 \approx 3.66 \times 10^{15}$ possible passphrases.

Offline: $N / 10^{10} = 365{,}615.84$ s ≈ 4.23 days.

Online: $N / 10 = 3.656 \times 10^{14}$ s ≈ 11.6 million years.

> **REALITY CHECK**
>
> These numbers assume random passwords. Real people choose predictable ones ("Tashkent2024!", "Qwerty123"). Dictionary and rule-based attacks try those first, so a human-chosen 12-character password can fall far faster than row 4 suggests. Lockouts and rate limits are what make the online column so slow — which is why stolen hash databases are so dangerous.

Part C — IAAA audit of one of your own accounts (15 min)

Pick one account you personally own — for example your university email, a cloud storage account or a messaging app. Sign in and explore its security settings. Record what you observe without recording any secret values.

IAAA element	Guiding question	What you observed
Identification	What label identifies you (email, phone number, username)? Is it public or secret?	Model answer — Google account: the email address identifies the account. It is an identifier, not a secret.
Authentication	Which factors are supported: something you know, have, are? Is MFA enabled? Which methods (SMS, authenticator app, passkey, hardware key)?	Password (something you know) plus a second factor such as an authenticator prompt/app or passkey (something you have). MFA can be enabled. Verify which methods are enabled on your own account.
Authorization	Can you see what third-party apps or devices are authorized to use this account? Can you revoke them?	Security settings show signed-in devices and connected third-party apps; access can be reviewed and revoked.
Accountability	Is there a sign-in history or activity log showing time, device and location? How far back does it go?	Recent security activity/sign-in information can show device, time and approximate location, supporting accountability.

IAAA element	Guiding question	What you observed
Recovery	How can the account be recovered if you forget the password? Could an attacker abuse that path?	Recovery can use a recovery email/phone and other verification. An attacker could abuse recovery if they control those channels, so they must also be secured.

Account type audited (no usernames): Google account — MODEL ANSWER; verify against your own account.
MFA before the lab: verify in your own settings.

Part D — Quick availability calculation (5 min)

A year has 8,760 hours. Calculate the maximum allowed downtime per year.

Availability target	Allowed downtime per year (show working)
99.5 %	8,760 × (1 − 0.995) = 43.8 hours ≈ 1 day 19 h 48 min
99.95 %	8,760 × (1 − 0.9995) = 4.38 hours ≈ 4 h 22 min 48 s

4. Analysis questions

Q1. A hash detects that a file changed, but it does not prevent the change. What else is needed to prevent or recover from unauthorized modification? (3 pts)
Access controls and least privilege are needed to prevent unauthorized changes. Backups, versioning and tested recovery procedures are needed to restore the correct file if a change still occurs.

Q2. If an attacker takes over a download website, they can replace both the file and the published checksum. What additional mechanism (hint: digital signatures, HTTPS, separate channels) protects against this? (3 pts)
Use a trusted digital signature that is verified with the publisher's public key. HTTPS helps protect the download channel, and publishing/verifying the checksum or signature through a separate trusted channel makes replacement of both file and checksum harder.

Q3. Using your Part B results, compare row 3 and row 5. What does this tell you about long passphrases versus short complex passwords? Which would you recommend in a company password policy, and why? (3 pts)
Row 3 (8 printable characters) has about 6.63×10^{15} combinations and an offline worst-case time of about 7.7 days. Row 5 (4 random Diceware words) has about 3.66×10^{15} combinations, so with exactly four words its

mathematical keyspace is slightly smaller. The important lesson is that length and randomness matter, and word-based passphrases are easier for people to remember. A company policy should prefer long, unique, randomly generated passwords or passphrases, together with MFA, rather than forcing short complicated patterns.

Q4. Why is an SMS code considered a weaker second factor than an authenticator app or a hardware key? Which threat category from the lecture does SIM-swapping belong to? (3 pts)

SMS is weaker because phone numbers can be hijacked through SIM-swapping, messages may be redirected, and SMS depends on the mobile network. Authenticator apps and hardware keys do not rely on control of the phone number; hardware security keys are also resistant to many phishing attacks. In the standard threat-category model, SIM-swapping is best treated as espionage/trespass because the attacker obtains unauthorized access by taking over an authentication channel.

Q5. The lecture showed an audit log where an “admin” account exported a customer file at 23:48. Explain why shared accounts destroy accountability, and name one control that fixes the problem. (3 pts)

With a shared 'admin' account, the log only shows that the shared account acted; it cannot prove which person performed the export. That destroys individual accountability. Give every administrator a unique named account, require MFA, and log privileged actions under that individual identity.

5. Deliverables and submission

- Completed tables for Parts A–D and answers to Q1–Q5.
- Save as PDF named Lab2_Surname1_Surname2.pdf and upload before the deadline.
- Do not include any passwords, codes or screenshots of security settings that reveal secrets.

Grading rubric

Criterion	Excellent	Adequate	Weak	Points
Part A — Hashing	All versions hashed; correct Y/N; avalanche and rename results explained	Most versions correct	Tool not working or table incomplete	25
Part B — Keyspace	All rows correct (±5%), sensible units, full working shown	Minor arithmetic or unit errors	Several rows wrong or missing	25
Part C — IAAA audit	All five elements observed and described precisely, no secrets disclosed	Some elements vague	Incomplete or discloses secrets	20
Part D — Availability	Both values correct with working	One correct	Missing	5
Questions Q1–Q5	Accurate and uses lecture vocabulary	Partly correct	Missing or off-topic	20
Presentation	Neat, complete, correctly named	Minor issues	Hard to read	5